CAHIER DES CHARGES OFFICIEL

PROJET D'EXAMEN : DÉPLOIEMENT D'UNE PWA DE TONTINE COLLABORATIVE

Application : TontiNaïs
Méthode : AI Fullstack Builder (AFB)
Rôle : Product Manager & Architecte
Statut : Version Validée & Blindage Sécurité Niveau Production (90%+)

1. Identité Visuelle et Branding

Le produit a été conçu pour inspirer la confiance, la sécurité financière et la modernité, tout en respectant les codes culturels de l'épargne solidaire (tontine).

- **Nom de l'application :** TontiNaïs (Contraction de Tontine et de Naïs, symbolisant la clarté, l'agilité et la confiance financière).
- **Slogan :** L'épargne solidaire, intelligente et sécurisée à portée de main.
- **Logo :** Représentation stylisée d'un pictogramme constitué de formes géométriques imbriquées avec le nom de l'application et le slogan.

Charte Colorimétrique Centralisée

Couleur	Code Hexadécimal	Signification Métier
Bleu Nuit (Primaire)	#1A365D	Confiance & Sécurité
Vert Émeraude (Secondaire)	#319795	Prospérité & Croissance
Or Solaire (Accent)	#ED8936	Dynamisme & Événements

2. L'Infrastructure de Base (Le Socle)

Conformément aux exigences du sujet d'examen, les environnements de test précédents sont écartés au profit d'une configuration rigoureuse et isolée :

- **Base de Données :** Connexion directe configurée via l'URI MongoDB Atlas de production : mongodb+srv://TontiNais:DabeJerome@tontinais.xdtbgyd.mongodb.net/?appName=TontiNais Restriction d'accès stricte via Whitelist IP et gestion des rôles minimaux requis.
- **Gestion de Version (GitHub) :** Dépôts dédiés pour le backend et le frontend.

3. Cahier des Charges & Vision Métier

3.1. Définition des Acteurs

Rôle	Périmètre & Responsabilités
Administrateur	Supervision globale de la plateforme, gestion des litiges majeurs, configuration des paramètres globaux, gestion multi-comptes des administrateurs et accès au Dashboard central de contrôle.
Chef de Tontine	Créateur et gestionnaire de son groupe de tontine. Définit les règles du cycle (montant de la cotisation, périodicité, nombre de participants), valide ou rejette les preuves de paiement de ses membres et orchestre la distribution des cagnottes.
Membre	Participant actif du groupe de tontine. Consulte le planning des tours, effectue ses versements via Mobile Money / Virement, téléverse ses reçus de paiement et suit l'état de son historique de cotisations.

3.2. Flux de Création d'un Groupe

- **Initialisation :** Le Chef de Tontine renseigne le nom du groupe, le montant de la cotisation unitaire, la devise, la fréquence et la date de démarrage.
- **Invitation :** Génération d'un lien d'invitation sécurisé unique et d'un code QR partageable.
- **Adhésion :** Le membre clique, s'inscrit/se connecte, et soumet une demande validée en un clic par le Chef de Tontine.

3.3. Flux de Paiement & Validation Manuelle

Utilisation obligatoire de l'API Cloudinary pour l'hébergement sécurisé des justificatifs de paiement.

- **Déclaration :** Saisie de la référence de transaction et téléversement du reçu. Envoi immédiat sur Cloudinary, stockage de l'URL sécurisée, statut initialisé à *En attente de validation*.
- **Validation / Rejet :** Le Chef de Tontine valide (passe à *Payé*) ou rejette avec motif (passe à *En retard / À corriger*).

3.4. Sécurité des Droits d'Accès & Gestion Multi-Compte Administrateur

- Authentification par JWT couplée à un middleware strict verifyAdmin.
- Collection MongoDB dédiée aux rôles (Super Admin, Modérateur Financier) avec hachage des mots de passe par bcrypt.
- Double verrou sur les routes sensibles du Backend : validité du JWT + contrôle des permissions.

3.5. Gestion des Cas Limites

- **Suspension Automatique :** Passage automatique en *Défaillant* en cas de dépassement de la date limite de grâce, restreignant l'accès aux nouveaux tours.
- **Fonds de Garantie / Remplacement :** Activation d'un membre suppléant ou absorption temporaire via un fonds de réserve.

4. Spécifications de Blindage Backend & Sécurité Maximale (Objectif 90%+)

Ce module exhaustif est destiné à Antigravity pour implémenter un durcissement de niveau production sur l'API Node.js / Express.

4.1. Analyse des Vecteurs d'Attaque (Menaces & Contournements)

Type d'Attaque	Mode Opératoire du Hacker / Malveillant	Impact Métier / Technique
Injections NoSQL (MongoDB)	Injection d'opérateurs MongoDB malveillants ($ne, $gt) dans les corps de requêtes JSON (ex: formulaires de login) pour contourner l'authentification.	Vol de comptes, usurpation d'identité de Chef de Tontine ou d'Administrateur.
Vol & Manipulation de JWT	Extraction du token stocké imprudemment, rejeu de requêtes expirées, ou attaque par force brute de la clé secrète (JWT Secret) si elle est faible.	Accès non autorisé aux routes financières, validation frauduleuse de cagnottes.
Attaques par Déni de Service (DoS / DDoS)	Saturation de l'API Node.js par un volume massif de requêtes non authentifiées ou requêtes lourdes (payloads volumineux).	Indisponibilité totale de la PWA TontiNaïs au moment des cotisations.
Broken Object Level Authorization (BOLA / IDOR)	Modification de l'ID d'un groupe ou d'un utilisateur dans l'URL/paramètres (ex: /groups/123 -> /groups/124) pour consulter ou modifier les données d'autrui.	Consultation de données financières confidentielles, détournement de tontines.
Attaques XSS & Injection de Code Client	Injection de scripts malveillants (JavaScript) dans les champs de texte (noms de groupes, motifs de rejet) s'ils sont reflétés sans échappement.	Vol de sessions utilisateur, exfiltration de cookies/tokens en clair.
Attaques CSRF & Cors Misconfigured	Exploitation de politiques CORS trop permissives permettant à des sites tiers	Actions non consenties réalisées à l'insu

Type d'Attaque	Mode Opératoire du Hacker / Malveillant	Impact Métier / Technique
	d'exécuter des requêtes authentifiées.	d'un administrateur connecté.
Contournement des Uploads de Fichiers (Cloudinary / Multer)	Envoi de fichiers exécutables (.exe, .php, scripts) déguisés en images de reçus de paiement.	Tentative d'exécution de code distant ou saturation du stockage Cloudinary.
Attaques par Force Brute (Rate Limiting absent)	Tests automatisés de milliers de combinaisons d'e-mails/mots de passe sur les routes de login.	Compromission des comptes administrateurs.

4.2. Technologies, Middlewares & Protocoles de Blindage Backend

Antigravity doit intégrer et configurer rigoureusement les modules suivants dans le projet Node.js / Express :

- **Sanitization NoSQL (express-mongo-sanitize) :** Nettoyage automatique de tous les objets req.body, req.query et req.params pour neutraliser les caractères spéciaux MongoDB ($ et .).
- **Sécurité des En-têtes HTTP (helmet) :** Configuration agressive des en-têtes HTTP de sécurité (Content Security Policy, X-Frame-Options, HSTS, désactivation de la signature X-Powered-By).
- **Limitation du Taux de Requêtes (express-rate-limit) :**
  - Rate limiter global sur l'API (ex: max 100 requêtes par 15 minutes par IP).
  - Rate limiter strict sur les routes d'authentification (ex: max 5 tentatives par 15 minutes) pour bloquer le bruteforcing.
- **Protection contre la Pollution de Paramètres (hpp) :** Prévention des attaques de pollution de paramètres HTTP (HTTP Parameter Pollution).
- **Validation de Schémas Stricts (express-validator ou Joi) :** Validation rigoureuse de chaque type de données entrantes (formats d'e-mails, structures de montants).
- **Gestion Fine des CORS (cors) :** Restriction absolue des origines autorisées avec interdiction des jokers * en production.
- **Hachage Robuste (bcrypt / argon2) :** Utilisation d'un facteur de coût élevé (salt rounds = 12) pour le stockage des mots de passe.
- **Gestion Sécurisée des JWT :** Signature avec une clé secrète longue et complexe, expiration courte des tokens d'accès couplée à des refresh tokens sécurisés.
- **Sécurisation des Uploads Fichiers :** Validation stricte du type MIME des reçus (uniquement images) via multer avant transmission à l'API Cloudinary.

5. Plan d'Orchestration Agentique (Rappel Workflow AFB)

Le développement de l'application sera exécuté rigoureusement sous l'environnement Antigravity selon les étapes suivantes :

1. **Initialisation :** Lancement du Prompt Maître de calibration configurant l'IA en mode "Architecte Senior".
2. **Génération Backend :** Fourniture de ce cahier des charges blindé, validation du plan d'exécution et développement progressif de l'API Node.js/Express (par blocs de 5 fichiers maximum avec prise en compte

intégrale de la sécurité).

3. **Déploiement Backend :** Hébergement sur Render, configuration de l'URI MongoDB Atlas mongodb+srv://TontiNais:DabeJerome@tontinais.xdtbgyd.mongodb.net/?appName=TontiNais, récupération de l'URL de production et configuration stricte des règles CORS.
4. **Génération Frontend :** Développement de l'interface React / Vite en local, connexion exclusive au Backend via les variables d'environnement (.env).